ADROSONIC BUILD | HACKATHON SUBMISSION NOTES

FR-1 Approach: File Ingestion & Sheet Discovery

How the Agentic SOV system reads a messy client workbook, finds the real data table, and picks the right sheet, without changing any client data.

Phase 1 of 4 · Scope: FR-1 + Agent 1 · 03 October 2026

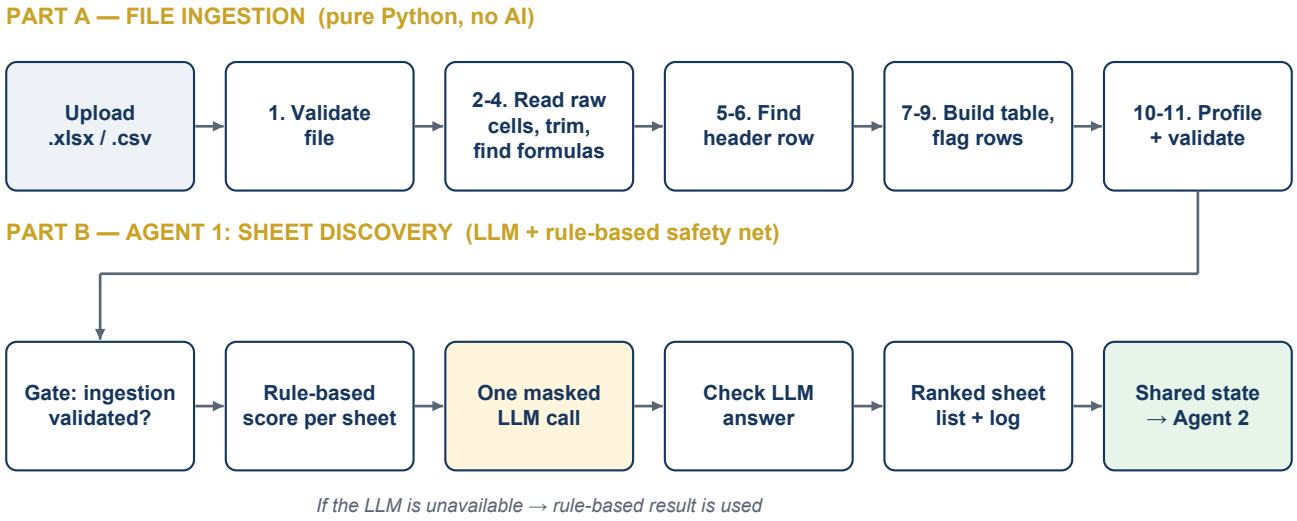
1. Summary

Phase 1 has two parts. **File Ingestion** is plain Python: it validates the upload, reads every cell exactly as stored, finds the header row wherever it sits, and builds a clean table per sheet. **Agent 1 (Sheet Discovery)** then decides which sheet holds the real property data: Python scores every sheet, one LLM call classifies and ranks them from a small masked summary, and Python checks the LLM's answer before anything uses it. All results go into one shared state object that the next agent reads.

One-line pitch: Python handles structure (reliable, fast, free); the LLM handles business judgement on a small summary; Python checks the LLM's answer. Phase 1 only reads and flags. It never changes, fills or deletes client data (C-01, C-02).

FR-1 requirement (problem statement)	How it is met
Accept .xlsx and .csv uploads through a web or desktop UI	Streamlit web page (`app.py`); .xlsx, .xlsm and .csv accepted
Support multi-sheet workbooks; identify the relevant tabs and headers automatically	Every sheet is profiled; Agent 1 ranks them and selects the Primary sheet
Handle header rows at any row position	Scored header detection over the first 30 rows (Section 4, Step 5)
Read files up to 5,000 rows without timeout or memory failure	5,000 × 40 workbook ingested in ~2.5 s; 8-sheet, 7,867-row sample in ~6 s
Produce a structured internal representation (e.g. a pandas DataFrame)	One pandas DataFrame per sheet, indexed by original Excel row number, held in a shared state

2. Pipeline at a glance



If the LLM is unavailable → rule-based result is used

3. Tools used

Tool	What it does for us
Python 3.12	Programming language
openpyxl	Opens Excel files; reads every cell, merged cells and hidden-sheet status
csv (built-in)	Reads CSV files (with our own encoding and delimiter detection)
zipfile + XML (built-in)	An .xlsx is a zip of XML files; we scan it to find formulas without saved results
pandas	Holds each final clean table as a DataFrame
pydantic	Defines the shared state object and validates the LLM's JSON answer
openai (GPT-4o by default)	Agent 1's single classification call; OpenAI or Azure OpenAI via environment variables
Streamlit	Minimal upload and review page
pytest	42 automated tests, including the 4 real sample files

4. Part A: File Ingestion (no AI)

Like receiving a messy paper form: check it really is a form, find where the table starts, and put sticky notes on anything odd, but never erase or rewrite anything.

Step 1: Check the file is acceptable (NFR-4)

Check	If it fails, the user sees
Extension must be .xlsx, .xlsm or .csv	"Not a supported file type. Please upload an .xlsx or .csv file."
File must not be 0 bytes	"The file is empty (0 bytes)."
File must be 25 MB or smaller	"File is X MB; the limit is 25 MB."
First 4 bytes must be a zip signature (every real .xlsx starts with PK)	"Not a valid Excel workbook or it is damaged."
Old .xls / password-protected signature detected	"Password-protected or old .xls. Please save it as an unprotected .xlsx."
A .csv that is really an Excel file	"Looks like an Excel workbook saved with a .csv extension."

The user never sees a crash or stack trace, only a plain-English message.

Step 2: Read every cell exactly as stored (C-01)

- **Excel:** openpyxl in *data_only* mode (formula results, not formula text). Every sheet is read, including hidden ones, and hidden status is recorded. Values are copied without conversion, so Zip `00802` stays `00802`.
- **CSV:** encoding tried in order UTF-8 (with BOM), UTF-8, Windows-1252. Delimiter chosen among `,` `;` `tab` `|` by consistency: the one giving the most lines with the same number of fields wins. (Python's built-in sniffer failed on CSVs with title lines above the header.)

Step 3: Cut off empty space

- Stop reading after **500 consecutive empty rows**. Example: H6D2 reports 4,786 used rows because of formatting; data ends at row 29.
- Trim empty columns on the right edge; pad rows to equal width.
- Hard cap of **50,000 rows** per sheet, with a note, so a giant file cannot crash the system.

Step 4: Detect formulas with no saved result

In all 4 sample files, every formula (about 1,500 cells) has no saved answer: the files were re-saved without recalculation, so totals and TIV columns read as empty. We scan the sheet XML for formula cells with no value and report them, e.g. *"'TIV' (34 cells, e.g. N7: =J7+K7+M7): a file artefact, not missing client data."* We do **not** calculate them: that would create data without approval.

Step 5: Find the header row (the core algorithm)

5a. Candidates: the first **30** non-empty rows. **5b. Skipped immediately:**

Skip rule	Why	Example
Fewer than 2 filled cells, or fewer than 30% of the widest row	Title or note line	"Statement of Values – Client X"
Less than 60% of filled cells are text	Headers are labels; numeric rows are data	1, 1, 250000, 1970, …
No non-empty rows below it	A header must have data under it	Last row of a sheet

"Text" means a non-empty string that is not a number ("$1,200", "45%" and "(500)" count as numbers), not a date, and not true/false.

5c. Five marks per remaining row (each 0 to 1, then weighted):

Mark	Plain meaning	Calculation	Weight
SOV words	Contains insurance terms (Zip, City, Year Built, TIV…)	matching cells ÷ 4, capped at 1	30%
Text share	Row is mostly words	text cells ÷ filled cells	20%
Contrast	Much wordier than the 10 rows below	row text share − average text share below (0 to 1)	20%
Coverage	Stretches across the table	filled cells ÷ filled cells of widest row	15%
Uniqueness	Labels do not repeat	distinct values ÷ filled cells	15%

```
score = 0.30·SOV + 0.20·Text + 0.20·Contrast + 0.15·Coverage + 0.15·Uniqueness
```

- **5d. Winner:** highest score; rows within 0.02 of the best → the earliest wins; best below **0.40** → "no header found" (we do not guess). The score is stored as header confidence, with a written reason.
- **5e. Word matching:** a list of the 17 target fields with alternative names (e.g. Year Built = yr built, yr blt, yob…) plus general SOV terms (TIV, deductible, flood, latitude…). Text is lower-cased, # → "no", & → "and", punctuation removed. Words of ≤3 letters (st, bi, occ) must be the whole cell; longer words may appear inside it; plurals accepted; target-field words beat general words.

Worked example: SOV_B4ID

Row	Content	Outcome
9	"ACCOUNT NAME:", "Secret Harbour…" (2 cells)	Skipped: too few cells (needs ≈ 8 of 27)
12	"Loc #", "Street", "City", "Zip", "Building", "Year Built"… (27 cells)	Scored → **0.89, header**
13	1, 1, address, St. Thomas, V.I., 00802, 250000, 1970…	Skipped: only 44% text (needs 60%)

Row 12: SOV words 18 → 1.00×0.30 = 0.30 · Text 27/27 → 0.20 · Contrast 100% vs 47% → 0.53×0.20 = 0.11 · Coverage 27/27 → 0.15 · Uniqueness 25/27 → 0.93×0.15 = 0.14 · **Total ≈ 0.89**

Reason shown to the user: *"Row 12 selected: 27 of 27 cells are text, 18 match SOV terms, the rows below are 47% text vs 100% in the header. Rows 1-11 contain a title/preamble block outside the table."*

Step 6: Combine two-row headers

When a group label sits above the header (a merged "Values" over "Buildings | Contents"), the rows are combined only if the row above has ≥ 2 different labels, is sparser than the header, and has merged cells spanning several columns. Result: "Values - Buildings" (K4T9). If the lower label already contains the upper one, only the lower is kept ("Account" over "Account Name" → "Account Name").

Step 7: Merged cells

Where	What we do	Why
Header area	Copy the label across the merged range	Every column needs a name; labels only, no data changed
Data area	Leave empty; record the range for review	Filling would invent values (C-02)

Step 8: Build the clean table

Situation	Action
Header has extra spaces / line breaks ("Street\nAddress")	Tidied to "Street Address" (original text kept)
Column has data but no header	Named Unnamed_Col_F (F = Excel column letter)
Duplicate header names	Second becomes Name__2, third Name__3
No header and no data	Ignored (no information lost)
Header but no data	Kept: it shows a missing field
Completely blank row inside the table	Skipped (no information lost)

- Every row keeps its **original Excel row number** (DataFrame index `source_row`), so the later audit log can point back to the client's file.
- An **untouched copy** of each raw sheet is kept alongside the clean table.

Step 9: Flag suspicious rows (never delete)

Flag	Rule	Example
total_row	One of the first 3 filled cells starts with "Total" / "Grand Total"	B4ID row 28
subtotal_row	Same, starting with "Subtotal"	"Subtotal TX"
repeated_header_row	Every filled cell equals its column header, and ≥ half the columns filled	Header pasted mid-table
notes_row	Exactly 1 filled cell, text, ≥ 40 characters, table wider than 2 columns	B4ID rows 30-34

Flagged rows stay in the table; removing them is a recommendation a human approves later (C-01).

Step 10: Profile each sheet

Measure	Meaning
Header row, confidence, reason	Where the header is and why
Rows / columns	Size of the table
% empty (overall and per column)	Empty = nothing, blank, or spaces only
% numeric columns	Columns where ≥ 80% of filled values are numbers
Target fields matched	Which of the 17 target fields the headers resemble
Row continuity	Longest gap-free run of rows ÷ total rows
Hidden, merged ranges, flagged rows, uncached formulas, notes	Structural facts for Agent 1 and the reviewer

Step 11: Validation gate (NFR-7)

Ingestion is stamped **validated** only if at least one sheet has a header and at least one data row. Otherwise the user sees "No data table could be found in this file" with what was found on each sheet. Agent 1 refuses to run without this stamp.

5. Part B: Agent 1, Sheet Intelligence & Discovery

Goal (from the problem statement): identify the workbook's authoritative data sheet while minimising false-positive sheet selections.

Step 1: Check the gate

If ingestion is not validated, Agent 1 stops with a clear error.

Step 2: Rule-based score per sheet (0 to 1)

Factor	Calculation	Weight
Target fields matched	matched fields ÷ 8, capped at 1	35%
Completeness	1 − share of empty cells	20%
Header confidence	from Part A, Step 5	15%
Row volume	data rows ÷ 20, capped at 1	15%
Row continuity	from Part A, Step 10	10%
Sheet-name hint	1 for SOV / Locations / Schedule / Property…; 0 for Deleted / Elsewhere / Glossary / Original / Autos…; else 0.5	5%

- Hidden sheets: −0.10.
- Automatic score 0 (Reject): no header found, fewer than 2 data rows, or fewer than 3 columns.

Step 3: Rule-based classification (fallback and yardstick)

Class	Rule
Reject	Hard-reject condition, score below 0.35, or no column resembles an SOV property field
Primary	The single top-scoring sheet, if its score is ≥ 0.55
Secondary	Everything else. A sheet with the same columns as the top sheet (≥ 70% overlap) is **not** auto-selected; it gets a note so the reviewer can add it

Lesson from the real data: SOV_Q8B3 has "Deleted Locations" and "Insured Elsewhere" tabs with exactly the same columns as the main sheet. Selecting them would put uninsured properties into the cleaned SOV, so matching columns alone never makes a sheet Primary.

Step 4: One LLM call for all sheets

- **Sent per sheet:** header list, row/column counts, % empty, mostly-empty columns, target fields matched, % numeric columns, rule-based score, structural notes, and up to 5 sample rows (plus 1 from the middle for large sheets), each value cut to 40 characters. At most 20 sheets.
- **Privacy:** columns that look like names, addresses, emails, phones, brokers or contacts are masked as `<TEXT len=18>`; emails and phone numbers are masked anywhere. The full data never leaves the machine.
- **Why one call:** the LLM can compare sheets with each other to rank them; it is cheaper and faster.
- **Settings:** temperature 0, JSON-only response, 30-second timeout.
- **Prompt:** defines Primary / Secondary / Reject and warns about traps: deleted, sold or elsewhere-insured locations; the client's original copy; prior-year comparisons; vehicle and equipment schedules.

```
{"sheets": [{"sheet_name": "...", "classification": "Primary|Secondary|Reject",
  "confidence": 0.94, "rank": 1, "reasoning": ["...", "..."]}]}
```

Step 5: Check the LLM's answer (anti-hallucination)

Problem	What happens
Response is not valid JSON in the required shape	Retry once, then use the rule-based result
LLM names a sheet that does not exist	Dropped and logged
LLM skips a real sheet	Filled in from the rule-based result
No sheet classified Primary	Best-scoring sheet promoted, marked needs review
LLM picks a sheet with score < 0.35, or one whose name says Deleted / Elsewhere / Notes…	Marked needs review
LLM rejects a sheet with score ≥ 0.70	Marked needs review
No API key, timeout or network error	Rule-based result used; reason logged

Step 6: Output

- Ranked sheet list: classification, confidence, reasoning, method (LLM or rule-based), rule-based score, needs-review flag (a superset of the example format in the problem statement).
- Selected Primary sheet(s), which Agent 2 will receive.
- A decision-log entry per sheet; Agent 1 stamps itself validated.

Step 7: Human override

The reviewer can change the selected sheet(s) in the UI; the change is logged with the reviewer's name.

6. Rules that apply to every step

Rule	How Phase 1 satisfies it
C-01 No auto-transformation	Phase 1 only reads and flags; no value is changed, filled or deleted
C-02 No hallucinated data	Merged data cells left empty; formulas without results not computed
C-04 / NFR-7 Shared state	One PipelineState object; each stage stamps itself validated; the next stage checks the stamp
C-05 / NFR-6 Explainability	Every decision (header picked, column renamed, row flagged, sheet classified) is logged with a plain-English reason and timestamp
NFR-4 Error handling	Friendly message for every bad file; the UI never shows a stack trace
Privacy / secrets	API keys only from environment variables; masked samples to the LLM; no raw data in logs

7. Results on the real sample files

File	Structure	Phase 1 result
SOV_B4ID	1 sheet; title block; header on row 12; Total row; merged notes rows	Header row 12 found; Total row and 5 notes rows flagged; Zip 00802 preserved
SOV_H6D2	1 sheet; header on row 7 with line breaks; formatting to row 4,786	Trimmed to 20 real rows; header row 7; uncached formulas reported
SOV_K4T9	7 sheets: summary pivot, YoY comparison, Locations, glossary, disclaimers, client original	"Locations" selected; client original ranked Secondary; boilerplate rejected
SOV_Q8B3	8 sheets (3 hidden): 864-row main sheet, Deleted / Insured Elsewhere tabs, vehicle fleet sheets	Only "23-24 Values" selected; Deleted / Elsewhere kept out; vehicle sheets rejected

Results above are from the rule-based path (no API key was configured). 42 automated tests pass, including 5 on these files.

8. Known limitations and next steps

- **LLM path tested with simulated responses only.** Run once with a real API key before the evaluation.
- **Formula results are not computed.** They are reported as a file problem; computing them would need an approved step. None of the affected columns is one of the 17 target fields, except two lump-sum rows in SOV_Q8B3.
- **Vocabulary was extended while studying the samples** (e.g. "# floor", "% sprinklered"). It is a fast first pass; Agent 2's semantic matching handles headers outside the list.
- **Simplification option:** header detection could be reduced to two rules (mostly text + most SOV words), with row flags and formula notes moved to Agent 3, at no loss on the sample files.
- **Next:** Agent 2 (schema mapping), Agent 3 (data quality + recommendations), approval UI, Agent 4 (transformation, Cleaned_SOV.xlsx, audit log), and an orchestrator linking the four agents.